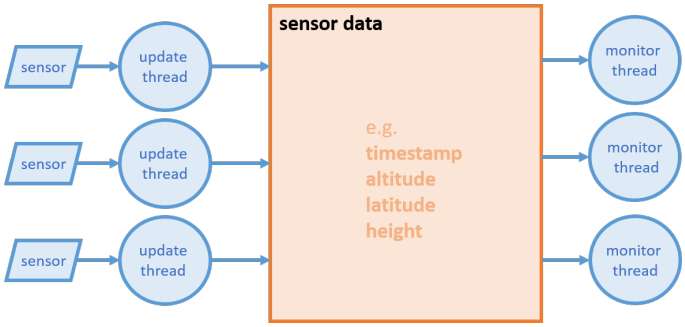
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Parallel Programming
Assignment 12: Lock-free sensor system
Spring Semester 2026

Lock Free Sensor System

This exercise is about the lock-free implementation of a data record. We consider the following scenario: Assume you have a number of sensors that deliver complex data. Retreiving the data from a sensor and in particular writing the data to some data structure cannot happen atomically. In this exercise we consider a sensor that delivers floating point data and an integer timestamp. You may want to think of a GPS sensor delivering, say, altitude, longitude and height together with the current time in milliseconds. In order to keep the system reliable and responsive, redundant data sources are installed, i.e. we have multiple sensors that provide the same kind of data.



Now, we assume we have a lot of concurrent reader threads that want to monitor the sensor data while we have a moderate number of writer threads. What we consider most important is that the reader threads do always read a consistent data set. Secondly, we require that only the newest sensor data are written to the sensor data object. This is where the timestamp becomes important.

Implementation

Implement two versions of the sensor data class:

a) One blocking version based on a readers-writers lock (*LockedSensors.java*).

b) A lock-free version (*LockFreeSensors.java*)

Hints:

a) Before you implement the readers-writers lock based version, start with a simple locked version in order to understand. Then try a readers-writers lock but be aware that the Java-implementation does not give fairness guarantees. What can this imply? In any case, you have the code from the lecture slides presenting a fair RW-Lock implementation.

b) The lock-free implementation solutions does NOT rely on mechanisms such as Double-Compare-And-Swap. Also it does not rely on a lazy update mechanism. Somehow you have to make sure that with a single reference update you change all data at once. How? (Hint: while each data object consists of multiple fields, and thus you cannot update all of them with a single CAS, the reference to the most up-to-date data object is a single value.)

Compare the efficiency of the two solutions. Experiment with different numbers of readers and writers and different timings in order to find out under which conditions which version is better. Start with the template that we provide.

Note: When testing your implementation using the provided Unit Tests make sure that you run the tests multiple times (i.e. 5-10 times). This increases the chance of finding an error in your implementation as the Unit Tests can succeed even with an invalid implementation due to randomness of scheduling.

Questions

a) In your lock-free solution, can it happen that a thread needs to continuously retry an operation, potentially never making progress, while other threads make progress? We still consider such algorithms lock-free. But it is clear that we could ask for an even stronger property: every thread always finishes in a bounded number of steps. Such algorithms are called wait-free.

b) Think about the solution you have provided: can you generalize this? Imagine, for example, the scenario of an expression tree that is updated by (a small amount of) writer threads sporadically but is used for evaluation by a huge number of reader threads. Can you use the same kind of mechanism?

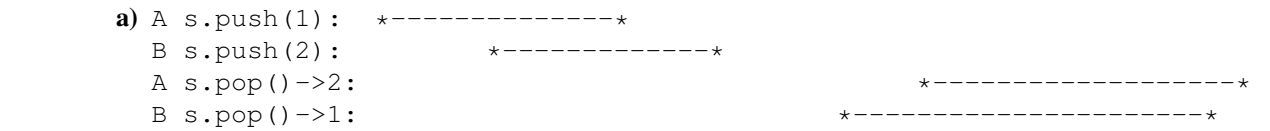
Linearizability and Sequential Consistency

In the following questions let A, B and C denote threads operating on a shared stack object as specified in the listing below.

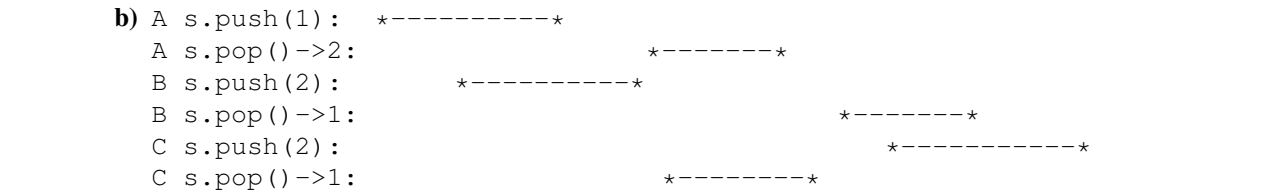
```
public interface Stack {
    /* pushes element v onto the stack */
    public void push(int v);

    /* removes the top element and returns it */
    public int pop();

    /* returns the top element */
    public int top();
}
```

Which of the following scenarios on the next page are lineraly consistent? Either mark the point of linearization or explain why it is not linearly consistent.

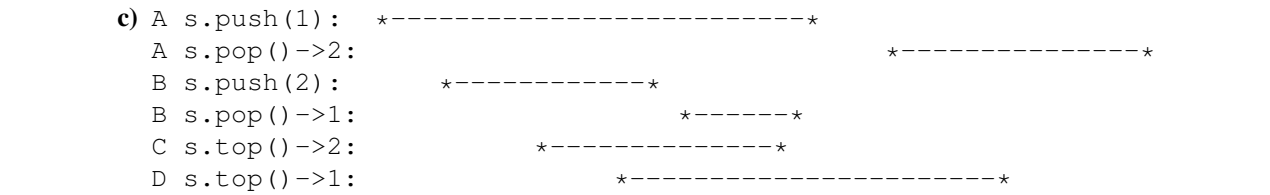
a)


A s.push(1):
B s.push(2):
A s.pop()->2:
B s.pop()->1:

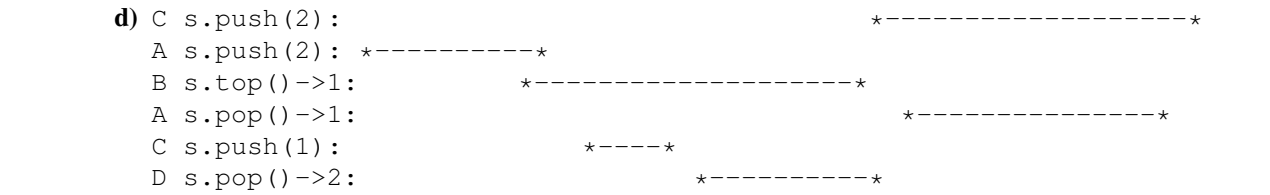
b)


A s.push(1):
A s.pop()->2:
B s.push(2):
B s.pop()->1:
C s.push(2):
C s.pop()->1:

c)


A s.push(1):
A s.pop()->2:
B s.push(2):
B s.pop()->1:
C s.top()->2:
D s.top()->1:

d)


C s.push(2):
A s.push(2):
B s.top()->1:
A s.pop()->1:
C s.push(1):
D s.pop()->2: